Looking at this output, it's easy to understand how generators work in Rails 3.0 and above. The scaffold generator doesn't actually generate anything, it just invokes others to do the work. This allows us to add/replace/remove any of those invocations. For instance, the scaffold generator invokes the scaffold_controller generator, which invokes erb, test_unit and helper generators. Since each generator has a single responsibility, they are easy to reuse, avoiding code duplication.

Our first customization on the workflow will be to stop generating stylesheet, JavaScript and test fixture files for scaffolds. We can achieve that by changing our configuration to the following:

```
config.generators do |g|
  g.orm             :active_record
  g.template_engine :erb
  g.test_framework  :test_unit, fixture: false
  g.stylesheets     false
  g.javascripts     false
end
```

If we generate another resource with the scaffold generator, we can see that stylesheet, JavaScript and fixture files are not created anymore. If you want to customize it further, for example to use DataMapper and RSpec instead of Active Record and TestUnit, it's just a matter of adding their gems to your application and configuring your generators.

To demonstrate this, we are going to create a new helper generator that simply adds some instance variable readers. First, we create a generator within the rails namespace, as this is where rails searches for generators used as hooks:

```
$ bin/rails generate generator rails/my_helper
      create  lib/generators/rails/my_helper
      create  lib/generators/rails/my_helper/my_helper_generator.rb
      create  lib/generators/rails/my_helper/USAGE
      create  lib/generators/rails/my_helper/templates
```

After that, we can delete both the `templates` directory and the `source_root` class method call from our new generator, because we are not going to need them. Add the method below, so our generator looks like the following:

```
# lib/generators/rails/my_helper/my_helper_generator.rb
class Rails::MyHelperGenerator < Rails::Generators::NamedBase
  def create_helper_file
    create_file "app/helpers/#{file_name}_helper.rb", <<-FILE
module #{class_name}Helper
  attr_reader :#{plural_name}, :#{plural_name.singularize}
end
    FILE
  end
end
```

We can try out our new generator by creating a helper for products:

```
$ bin/rails generate my_helper products
      create  app/helpers/products_helper.rb
```

And it will generate the following helper file in `app/helpers`:

```
module ProductsHelper
  attr_reader :products, :product
end
```

Which is what we expected. We can now tell scaffold to use our new helper generator by editing `config/application.rb` once again:

```
config.generators do |g|
  g.orm             :active_record
  g.template_engine :erb
```